

Chapter 24 — OOP Architecture & Core Learning Roadmap

OOP (Object-Oriented Programming) হলো আধুনিক সফটওয়্যার ইঞ্জিনিয়ারিংয়ের মূল ভিত্তি। বিশেষ করে **React, Node.js, Express, NestJS, এবং large-scale enterprise application** ডেভেলপমেন্টে পরিষ্কার ডেটা মডেলিং এবং মডুলার কোড স্ট্রাকচার সাজাতে ক্লাস এবং অবজেক্টের কনসেপ্ট অপরিহার্য। এই চ্যাপ্টারে আমরা একদম গ্রাউন্ড লেভেল থেকে ইন্টারনাল প্রোটোটাইপ মেকানিজম পর্যন্ত নিখুঁতভাবে শিখব।

- ✓ OOP কী এবং কেন দরকার
- ✓ Object-oriented programming-এর 4 pillars
- ✓ Class কী
- ✓ Object কী
- ✓ Constructor
- ✓ this
- ✓ Instance properties
- ✓ Instance methods
- ✓ Static properties/methods
- ✓ Getters & Setters
- ✓ extends
- ✓ super()
- ✓ Method overriding
- ✓ Inheritance
- ✓ Encapsulation
- ✓ Private fields #
- ✓ Polymorphism
- ✓ Abstraction
- ✓ Composition
- ✓ instanceof
- ✓ Class vs Constructor Function
- ✓ Prototype-এর সাথে Class-এর সম্পর্ক
- ✓ Real-world project architecture
- ✓ Common mistakes
- ✓ Interview questions

Part 01 — OOP Fundamentals, Objects & Class Syntax (24.1 – 24.10)

24.1 OOP কী?

OOP = Object-Oriented Programming এটি এমন একটি programming approach যেখানে program-এর data এবং সেই data-এর সাথে কাজ করার functions-কে একসাথে **object** হিসেবে organize করা হয়। সহজভাবে: > **Data + Behavior = Object**

♦ Real-Life Example

ধরো একটি **Student**! Student-এর data:

► CONSOLE OUTPUT

OUTPUT

```
name  
id  
department  
cgpa
```

Student-এর behavior:

► CONSOLE OUTPUT

OUTPUT

```
study()  
attendClass()  
takeExam()
```

JavaScript-এ আমরা এগুলো একটি object/class-এর মাধ্যমে organize করতে পারি।

24.2 OOP কেন দরকার?

ছোট program-এ সাধারণ function এবং object দিয়েই কাজ করা যায়। কিন্তু বড় application হলে সমস্যা হয়। ধরো university system:

► CONSOLE OUTPUT

OUTPUT

```
Student
Teacher
Admin
Course
Department
Attendance
Result
Payment
```

প্রত্যেকটির আলাদা data এবং behavior আছে। OOP ব্যবহার করলে code:

- organized হয়
- reusable হয়
- maintain করা সহজ হয়
- বড় application manage করা সহজ হয়
- inheritance ব্যবহার করা যায়
- encapsulation করা যায়

24.3 OOP-এর 4 Pillars

OOP-এর চারটি মূল concept:

Pillar	সহজ অর্থ
Encapsulation	Data + Methods একসাথে রাখা এবং access control করা
Abstraction	প্রয়োজনীয় অংশ দেখানো, internal complexity hide করা
Inheritance	একটি class থেকে অন্য class-এর বৈশিষ্ট্য নেওয়া
Polymorphism	একই method/interface বিভিন্নভাবে কাজ করা

মনে রাখবে: > **E-A-I-P Encapsulation** → **Abstraction** → **Inheritance** → **Polymorphism**

24.4 Object কী?

Object হলো key-value pair-এর collection।

JAVASCRIPT

JAVASCRIPT

```
const student = {
  name: "Shariar",
  age: 23,
  department: "CSE",

  study() {
    console.log("Student is studying");
  }
};
```

এখানে:

► CONSOLE OUTPUT

OUTPUT

```
name
age
department
```

হলো data। আর:

JAVASCRIPT

JAVASCRIPT

```
study()
```

হলো behavior।

24.5 Class কী?

Class হলো object তৈরির একটি **blueprint/template**।

♦ Real-Life Example

একটি বাড়ির blueprint ধরো:

► CONSOLE OUTPUT

OUTPUT

```
Bedrooms  
Bathrooms  
Kitchen  
Garage
```

Blueprint নিজে বাড়ি নয়। Blueprint থেকে অনেক বাড়ি বানানো যায়। ঠিক একইভাবে:

JAVASCRIPT

JAVASCRIPT

```
class Student {  
}
```

হলো blueprint। আর:

JAVASCRIPT

JAVASCRIPT

```
const student1 = new Student();  
const student2 = new Student();
```

হলো সেই blueprint থেকে তৈরি objects।

24.6 Basic Class Syntax

JAVASCRIPT

JAVASCRIPT

```
class Student {  
}
```

Object তৈরি:

JAVASCRIPT

JAVASCRIPT

```
const student1 = new Student();
```

এখানে:

🕒 CONCEPT ARCHITECTURE

```
class → blueprint  
Student → class name  
new → নতুন object তৈরি  
student1 → object/instance
```

24.7 Constructor

Class-এর ভিতরে **constructor()** একটি special method। Object তৈরি হওয়ার সময় এটি automatically execute হয়।

JAVASCRIPT

JAVASCRIPT

```
class Student {  
  constructor() {  
    console.log("Student object created");  
  }  
}  
  
const student1 = new Student();
```

► CONSOLE OUTPUT

OUTPUT

```
Student object created
```

কারণ:

JAVASCRIPT

JAVASCRIPT

```
new Student();
```

চললে constructor automatically call হয়।

24.8 Constructor Parameters

এখন student-এর information পাঠাই।

JAVASCRIPT

JAVASCRIPT

```
class Student {  
  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
}
```

Object:

JAVASCRIPT

JAVASCRIPT

```
const student1 = new Student("Shariar", 23);  
  
console.log(student1.name);  
console.log(student1.age);
```

► CONSOLE OUTPUT

OUTPUT

```
Shariar  
23
```

24.9 this কী?

OOP-এ this অত্যন্ত গুরুত্বপূর্ণ।

JAVASCRIPT

JAVASCRIPT

```
class Student {  
  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
}
```

এখানে:

JAVASCRIPT

JAVASCRIPT

```
this.name
```

মানে: > যে object তৈরি হচ্ছে, সেই object-এর name property। যখন:

JAVASCRIPT

JAVASCRIPT

```
const student1 = new Student("Shariar", 23);
```

তখন:

JAVASCRIPT

JAVASCRIPT

```
this
```

মূলত student1 object-কে refer করছে। # 🧠 সহজভাবে

JAVASCRIPT

JAVASCRIPT

```
this.name = name;
```

মানে:

► CONSOLE OUTPUT

OUTPUT

```
object-এর name = parameter-এর name
```

24.10 Multiple Objects

একটি class থেকে অনেক object তৈরি করা যায়।

JAVASCRIPT

JAVASCRIPT

```
class Student {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
}  
  
const student1 = new Student("Shariar", 23);  
const student2 = new Student("Rahim", 22);  
const student3 = new Student("Karim", 24);  
  
console.log(student1);  
console.log(student2);  
console.log(student3);
```

প্রতিটি object-এর নিজস্ব data থাকবে।

Part 02 — Methods, Properties, Accessors & Encapsulation (24.11 – 24.23)

24.11 Instance Method

Class-এর ভিতরে function লিখলে সেটাকে সাধারণত **method** বলা হয়।

JAVASCRIPT

JAVASCRIPT

```
class Student {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  introduce() {  
    console.log(`My name is ${this.name}`);  
  }  
}
```

ব্যবহার:

JAVASCRIPT

JAVASCRIPT

```
const student = new Student("Shariar", 23);  
  
student.introduce();
```

► CONSOLE OUTPUT

OUTPUT

My name is Shariar

24.12 Multiple Methods

JAVASCRIPT

JAVASCRIPT

```
class Student {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  introduce() {  
    console.log(`My name is ${this.name}`);  
  }  
  
  study() {  
    console.log(`${this.name} is studying`);  
  }  
  
  sleep() {  
    console.log(`${this.name} is sleeping`);  
  }  
}  
  
const student = new Student("Shariar", 23);  
  
student.introduce();  
student.study();  
student.sleep();
```

► CONSOLE OUTPUT

OUTPUT

```
My name is Shariar  
Shariar is studying  
Shariar is sleeping
```

24.13 Real-Life Example: Bank Account

JAVASCRIPT

JAVASCRIPT

```
class BankAccount {  
  constructor(owner, balance) {  
    this.owner = owner;  
    this.balance = balance;  
  }  
  
  deposit(amount) {  
    this.balance += amount;  
  }  
  
  withdraw(amount) {  
    if (amount > this.balance) {  
      console.log("Insufficient balance");  
      return;  
    }  
  
    this.balance -= amount;  
  }  
  
  showBalance() {  
    console.log(`Balance: ${this.balance}`);  
  }  
}  
  
const account = new BankAccount("Shariar", 10000);  
  
account.deposit(5000);  
account.withdraw(2000);  
account.showBalance();
```

► CONSOLE OUTPUT

OUTPUT

Balance: 13000

কারণ:

► CONSOLE OUTPUT

OUTPUT

10000 + 5000 - 2000
= 13000

24.14 Instance Properties

যে properties প্রতিটি object-এর জন্য আলাদা হয় সেগুলো instance properties।

JAVASCRIPT

JAVASCRIPT

```
class User {  
  constructor(name, email) {  
    this.name = name;  
    this.email = email;  
  }  
}
```

JAVASCRIPT

JAVASCRIPT

```
const user1 = new User("Shariar", "shariar@example.com");  
const user2 = new User("Rahim", "rahim@example.com");
```

এখানে:

🕒 CONCEPT ARCHITECTURE

user1.name → Shariar
user2.name → Rahim

24.15 Static Properties

Static property class-এর সাথে associated থাকে, individual object-এর সাথে নয়।

JAVASCRIPT

JAVASCRIPT

```
class User {  
  
  static role = "Student";  
  
}
```

Access:

JAVASCRIPT

JAVASCRIPT

```
console.log(User.role);
```

► CONSOLE OUTPUT

OUTPUT

Student

কিন্তু:

JAVASCRIPT

JAVASCRIPT

```
const user = new User();  
  
console.log(user.role);
```

এটি `undefined` হবে কারণ `role` object-এর property নয়।

24.16 Static Method

Static method object দিয়ে নয়, class দিয়ে call করা হয়।

JAVASCRIPT

JAVASCRIPT

```
class MathHelper {  
  
  static add(a, b) {  
    return a + b;  
  }  
  
}  
  
console.log(MathHelper.add(10, 20));
```

► CONSOLE OUTPUT

OUTPUT

30

এটি:

JAVASCRIPT

JAVASCRIPT

```
MathHelper.add()
```

দিয়ে call করতে হবে।

24.17 Static Method Real-Life Example

JAVASCRIPT

JAVASCRIPT

```
class User {  
  
  constructor(name) {  
    this.name = name;  
  }  
  
  static createGuest() {  
    return new User("Guest");  
  }  
  
}  
  
const guest = User.createGuest();  
  
console.log(guest.name);
```

► CONSOLE OUTPUT

OUTPUT

Guest

এখানে `createGuest()` একটি utility/factory-like static method।

24.18 Getter

Getter দিয়ে property-এর মতো syntax ব্যবহার করে method-এর result পাওয়া যায়।

JAVASCRIPT

JAVASCRIPT

```
class Rectangle {  
  constructor(width, height) {  
    this.width = width;  
    this.height = height;  
  }  
  
  get area() {  
    return this.width * this.height;  
  }  
}  
  
const rectangle = new Rectangle(10, 5);  
  
console.log(rectangle.area);
```

► CONSOLE OUTPUT

OUTPUT

50

খেয়াল করো:

JAVASCRIPT

JAVASCRIPT

```
rectangle.area
```

এখানে `()` নেই। কারণ এটি getter।

24.19 Setter

Setter দিয়ে property set করার সময় custom logic চালানো যায়।

JAVASCRIPT

JAVASCRIPT

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
  
  set username(value) {  
    this.name = value.trim();  
  }  
}  
  
const user = new User("Shariar");  
  
user.username = " Ripon ";  
  
console.log(user.name);
```

► CONSOLE OUTPUT

OUTPUT

Ripon

24.20 Getter + Setter Together

JAVASCRIPT

JAVASCRIPT

```
class Person {  
  
  constructor(name) {  
    this._name = name;  
  }  
  
  get name() {  
    return this._name;  
  }  
  
  set name(value) {  
  
    if (value.length < 3) {  
      throw new Error("Name is too short");  
    }  
  
    this._name = value;  
  }  
}  
  
const person = new Person("Shariar");  
console.log(person.name);  
person.name = "Ripon";  
console.log(person.name);
```

24.21 Encapsulation

Encapsulation মানে data এবং behavior একসাথে রাখা এবং data access/control করা। ধরো Bank Account:

JAVASCRIPT

JAVASCRIPT

```
class BankAccount {  
  
  constructor(balance) {  
    this.balance = balance;  
  }  
  
}
```

এখানে বাইরে থেকে সরাসরি:

JAVASCRIPT

JAVASCRIPT

```
account.balance = -500000;
```

করা সম্ভব। এটি undesirable হতে পারে। Private field ব্যবহার করা যায়।

24.22 Private Fields

Modern JavaScript-৯:

JAVASCRIPT

JAVASCRIPT

```
class BankAccount {  
    #balance;  
  
    constructor(balance) {  
        this.#balance = balance;  
    }  
  
    getBalance() {  
        return this.#balance;  
    }  
}  
  
const account = new BankAccount(10000);  
  
console.log(account.getBalance());
```

► CONSOLE OUTPUT

OUTPUT

10000

কিন্তু:

JAVASCRIPT

JAVASCRIPT

```
console.log(account.#balance);
```

এটি **SyntaxError** হবে কারণ `#balance` private।

24.23 Private Method

Methods-ও private করা যায়।

JAVASCRIPT

JAVASCRIPT

```
class BankAccount {  
    #balance = 0;  
  
    #validateAmount(amount) {  
        return amount > 0;  
    }  
  
    deposit(amount) {  
        if (!this.#validateAmount(amount)) {  
            throw new Error("Invalid amount");  
        }  
  
        this.#balance += amount;  
    }  
  
    getBalance() {  
        return this.#balance;  
    }  
}
```

এটি encapsulation-এর ভালো example।

24.24 Inheritance

Inheritance মানে একটি class অন্য class-এর properties এবং methods ব্যবহার করতে পারবে। ধরো:

🕒 CONCEPT ARCHITECTURE

Animal
↓
Dog

Dog হলো Animal-এর একটি type।

24.25 extends

JAVASCRIPT

JAVASCRIPT

```
class Animal {  
  eat() {  
    console.log("Animal is eating");  
  }  
}  
  
class Dog extends Animal {  
  bark() {  
    console.log("Dog is barking");  
  }  
}  
  
const dog = new Dog();  
  
dog.eat();  
dog.bark();
```

▶ CONSOLE OUTPUT

OUTPUT

Animal is eating
Dog is barking

Dog নিজের method:

▶ CONSOLE OUTPUT

OUTPUT

bark()

পেয়েছে। Animal থেকে পেয়েছে:

▶ CONSOLE OUTPUT

OUTPUT

eat()

24.26 super()

Child class-এর constructor থাকলে parent constructor call করতে `super()` ব্যবহার করতে হয়।

JAVASCRIPT

JAVASCRIPT

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
}  
  
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name);  
    this.breed = breed;  
  }  
}  
  
const dog = new Dog("Tommy", "German Shepherd");  
  
console.log(dog.name);  
console.log(dog.breed);
```

► CONSOLE OUTPUT

OUTPUT

```
Tommy  
German Shepherd
```

24.27 super.method()

Parent class-এর method call করতেও `super` ব্যবহার করা যায়।

JAVASCRIPT

JAVASCRIPT

```
class Animal {  
  speak() {  
    console.log("Animal makes a sound");  
  }  
}  
  
class Dog extends Animal {  
  speak() {  
    super.speak();  
    console.log("Dog barks");  
  }  
}  
  
const dog = new Dog();  
  
dog.speak();
```

► CONSOLE OUTPUT

OUTPUT

```
Animal makes a sound  
Dog barks
```

24.28 Method Overriding

Child class parent-এর method নিজের মতো করে redefine করতে পারে।

JAVASCRIPT

JAVASCRIPT

```
class Animal {  
  speak() {  
    console.log("Animal sound");  
  }  
}  
  
class Dog extends Animal {  
  speak() {  
    console.log("Woof!");  
  }  
}  
  
const dog = new Dog();  
  
dog.speak();
```

► CONSOLE OUTPUT

OUTPUT

Woof!

এটাই method overriding।

24.29 Polymorphism

Poly = many Morph = forms অর্থাৎ: > একই interface/method বিভিন্ন object-এ বিভিন্ন behavior দিতে পারে।

JAVASCRIPT

JAVASCRIPT

```
class Animal {  
  speak() {  
    console.log("Animal sound");  
  }  
}  
  
class Dog extends Animal {  
  speak() {  
    console.log("Woof!");  
  }  
}  
  
class Cat extends Animal {  
  speak() {  
    console.log("Meow!");  
  }  
}  
  
const animals = [  
  new Dog(),  
  new Cat()  
];  
  
animals.forEach(animal => {  
  animal.speak();  
});
```

► CONSOLE OUTPUT

OUTPUT

Woof!
Meow!

একই:

JAVASCRIPT

JAVASCRIPT

```
animal.speak()
```

কিন্তু result আলাদা। এটাই polymorphism-এর basic example।

24.30 instanceof

কোন object কোনো class-এর instance কিনা check করতে:

JAVASCRIPT

JAVASCRIPT

```
class Student {}  
  
const student = new Student();  
  
console.log(student instanceof Student);
```

► CONSOLE OUTPUT

OUTPUT

true

আর:

JAVASCRIPT

JAVASCRIPT

```
console.log(student instanceof Array);
```

Output:

► CONSOLE OUTPUT

OUTPUT

false

24.31 Class + Private Data + Methods

একটি realistic example:

JAVASCRIPT

JAVASCRIPT

```
class BankAccount {  
  
  #balance;  
  
  constructor(owner, initialBalance = 0) {  
    this.owner = owner;  
    this.#balance = initialBalance;  
  }  
  
  deposit(amount) {  
    if (amount ≤ 0) {  
      throw new Error("Invalid deposit amount");  
    }  
  
    this.#balance += amount;  
  }  
  
  withdraw(amount) {  
    if (amount > this.#balance) {  
      throw new Error("Insufficient balance");  
    }  
  
    this.#balance -= amount;  
  }  
  
  getBalance() {  
    return this.#balance;  
  }  
}  
  
const account = new BankAccount("Shariar", 10000);  
  
account.deposit(5000);  
account.withdraw(2000);  
  
console.log(account.getBalance());
```

Output:

► CONSOLE OUTPUT

OUTPUT

13000

এখানে একসাথে ব্যবহার হয়েছে:

- Class
- Constructor
- `this`
- Private field
- Methods
- Encapsulation
- Error handling

24.32 Abstraction

Abstraction মানে: > জটিল internal implementation hide করে user-কে প্রয়োজনীয় interface দেওয়া।

♦ Real-Life Example

তুমি ATM ব্যবহার করো:

🕒 CONCEPT ARCHITECTURE

```
Insert Card
  ↓
Enter PIN
  ↓
Withdraw
```

তুমি জানো না ATM-এর ভিতরে:

► CONSOLE OUTPUT

OUTPUT

```
Database
Bank Server
Encryption
Transaction System
Network
```

কীভাবে কাজ করছে। এটাই abstraction-এর ধারণা। JavaScript example:

JAVASCRIPT

JAVASCRIPT

```
class ATM {

  withdraw(amount) {

    this.#verifyAccount();
    this.#checkBalance();
    this.#processTransaction();

    console.log(`Withdrawn: ${amount}`);
  }

  #verifyAccount() {
    console.log("Account verified");
  }

  #checkBalance() {
    console.log("Balance checked");
  }

  #processTransaction() {
    console.log("Transaction processed");
  }

}

const atm = new ATM();

atm.withdraw(5000);
```

User শুধু:

JAVASCRIPT

JAVASCRIPT

```
atm.withdraw(5000);
```

ব্যবহার করছে। Internal process hidden।

24.33 Composition

সবসময় inheritance ব্যবহার করতে হবে এমন নয়। Composition মানে ছোট ছোট object/component combine করে বড় behavior তৈরি করা। ধরো:

JAVASCRIPT

JAVASCRIPT

```
const engine = {
  start() {
    console.log("Engine started");
  }
};

const musicSystem = {
  playMusic() {
    console.log("Music playing");
  }
};

const car = {
  engine,
  musicSystem
};

car.engine.start();
car.musicSystem.playMusic();
```

এখানে car-এর:

► CONSOLE OUTPUT

OUTPUT

```
Engine
Music System
```

আলাদা components! # 🧠 Inheritance বনাম Composition

Inheritance	Composition
IS-A relationship	HAS-A relationship
Dog IS-A Animal	Car HAS-A Engine
extends	objects/components combine
Tight coupling হতে পারে	বেশি flexible হতে পারে

◆ Example

► CONSOLE OUTPUT

OUTPUT

```
Dog IS-A Animal
```

Inheritance! কিন্তু:

► CONSOLE OUTPUT

OUTPUT

```
Car HAS-A Engine
Car HAS-A GPS
Car HAS-A MusicSystem
```

Composition!

24.34 Class vs Object

Class	Object
Blueprint	বাস্তব instance
Template	Created entity
<code>class Student</code>	<code>new Student()</code>
অনেক object তৈরি করতে পারে	একটি নির্দিষ্ট instance

উদাহরণ:

JAVASCRIPT

JAVASCRIPT

```
class Student {}
```

Class

JAVASCRIPT

JAVASCRIPT

```
const student = new Student();
```

Object

24.35 Class vs Constructor Function

Modern JavaScript-এ `class` বেশি readable। আগে constructor function ব্যবহার করা হতো।

◆ Constructor Function

JAVASCRIPT

JAVASCRIPT

```
function Student(name) {
  this.name = name;
}

Student.prototype.introduce = function () {
  console.log(`I am ${this.name}`);
};

const student = new Student("Shariar");

student.introduce();
```

◆ Class Version

JAVASCRIPT

JAVASCRIPT

```
class Student {
  constructor(name) {
    this.name = name;
  }

  introduce() {
    console.log(`I am ${this.name}`);
  }
}

const student = new Student("Shariar");

student.introduce();
```

দুটো conceptually related।

24.36 গুরুত্বপূর্ণ: Class আসলে Prototype-এর উপর কাজ করে

এটা খুব গুরুত্বপূর্ণ। তুমি Chapter 11-এ Prototype পড়েছো। JavaScript-এর:

JAVASCRIPT

JAVASCRIPT

```
class Student {  
  
  introduce() {  
    console.log("Hello");  
  }  
  
}
```

এর methods সাধারণত prototype-এ থাকে। অর্থাৎ:

🕒 CONCEPT ARCHITECTURE

```
Class syntax  
↓  
Prototype-based mechanism
```

JavaScript মূলত prototype-based language। `class` হলো cleaner syntax/abstraction।

24.37 Prototype Check

JAVASCRIPT

JAVASCRIPT

```
class Student {  
  
  introduce() {  
    console.log("Hello");  
  }  
  
}  
  
const student = new Student();  
  
console.log(  
  Object.getPrototypeOf(student) === Student.prototype  
);
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

true

24.38 Class Fields

Modern JavaScript-এ class-এর মধ্যে directly field declare করা যায়।

JAVASCRIPT

JAVASCRIPT

```
class Student {  
  
  name = "Unknown";  
  age = 0;  
  
}  
  
const student = new Student();  
  
console.log(student.name);  
console.log(student.age);
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

Unknown
0

24.39 Private Class Field

JAVASCRIPT

JAVASCRIPT

```
class User {  
  #password;  
  
  constructor(password) {  
    this.#password = password;  
  }  
  
  checkPassword(password) {  
    return this.#password === password;  
  }  
}  
  
const user = new User("12345");  
  
console.log(user.checkPassword("12345"));  
console.log(user.checkPassword("wrong"));
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
true  
false
```

Password সরাসরি বাইরে access করা যাবে না।

JAVASCRIPT

JAVASCRIPT

```
class Product {  
  
  constructor(name, price, stock) {  
    this.name = name;  
    this.price = price;  
    this.stock = stock;  
  }  
  
  getTotalPrice(quantity) {  
  
    if (quantity > this.stock) {  
      throw new Error("Not enough stock");  
    }  
  
    return this.price * quantity;  
  }  
  
  sell(quantity) {  
  
    if (quantity > this.stock) {  
      throw new Error("Not enough stock");  
    }  
  
    this.stock -= quantity;  
  }  
  
  showProduct() {  
    console.log(  
      `${this.name} - ₺${this.price} - Stock: ${this.stock}`  
    );  
  }  
}  
  
const laptop = new Product("Laptop", 60000, 10);  
  
laptop.showProduct();  
  
console.log(  
  "Total:",  
  laptop.getTotalPrice(2)  
);  
  
laptop.sell(2);  
  
laptop.showProduct();
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Laptop - ₺60000 - Stock: 10  
Total: 120000  
Laptop - ₺60000 - Stock: 8
```

24.41 Inheritance in E-commerce

ধরো:

🕒 CONCEPT ARCHITECTURE

```
Product
  ↓
ElectronicProduct
  ↓
Laptop
```

JAVASCRIPT

JAVASCRIPT

```
class Product {
  constructor(name, price) {
    this.name = name;
    this.price = price;
  }

  showInfo() {
    console.log(`${this.name}: ₳${this.price}`);
  }
}

class ElectronicProduct extends Product {
  constructor(name, price, warranty) {
    super(name, price);
    this.warranty = warranty;
  }

  showWarranty() {
    console.log(`Warranty: ${this.warranty} months`);
  }
}

const laptop = new ElectronicProduct(
  "Laptop",
  60000,
  24
);

laptop.showInfo();
laptop.showWarranty();
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

```
Laptop: ₳60000
Warranty: 24 months
```

24.42 OOP Practical Architecture

একটি বড় project-এ এমন structure দেখা যেতে পারে:

📁 CONCEPT ARCHITECTURE

```
src/
├── models/
│   ├── User.js
│   ├── Product.js
│   └── Order.js
├── services/
│   ├── AuthService.js
│   ├── PaymentService.js
│   └── ProductService.js
├── controllers/
│   ├── UserController.js
│   └── ProductController.js
└── app.js
```

উদাহরণ:

JAVASCRIPT

JAVASCRIPT

```
class User {

  constructor(name, email) {
    this.name = name;
    this.email = email;
  }

  getProfile() {
    return {
      name: this.name,
      email: this.email
    };
  }
}

export default User;
```

এভাবে application-এর বিভিন্ন responsibility আলাদা করা যায়।

24.43 OOP + API Data

ধরো API থেকে user data পেলাম:

JAVASCRIPT

JAVASCRIPT

```
const data = {
  id: 1,
  name: "Shariar",
  email: "shariar@example.com"
};
```

Class দিয়ে model:

JAVASCRIPT

JAVASCRIPT

```
class User {

  constructor(data) {
    this.id = data.id;
    this.name = data.name;
    this.email = data.email;
  }

  getDisplayName() {
    return this.name;
  }
}

const user = new User(data);

console.log(user.getDisplayName());
```

Output:

► CONSOLE OUTPUT

OUTPUT

Shariar

এটি বড় application-এ usefull

Part 05 — Common Mistakes, Edge Cases & Mental Models (24.44 – 24.49)

24.44 Common Mistake: new ভুলে যাওয়া

Class constructor কখনো `new` কীওয়ার্ড ছাড়া সরাসরি ফাংশনের মতো কল করা যায় না। এভাবে কল করলে `TypeError` ছুঁড়ে দেয়।

ভুল পদ্ধতি: NEW কীওয়ার্ড ছাড়া কল

✗ ERROR / INVALID

```
class Student {}

const student = Student();
```

✗ RUNTIME ERROR

TYPEERROR

TypeError: Class constructor Student cannot be invoked without 'new'

সঠিক সমাধান: অবজেক্ট ইনস্ট্যান্স তৈরি করতে সর্বদা `new` ব্যবহার করুন।

সঠিক পদ্ধতি: NEW ব্যবহার

✓ CORRECT

```
class Student {}

const student = new Student();
```

24.45 Common Mistake: `super()` ভুলে যাওয়া

Inheritance ব্যবহার করার সময় চাইল্ড ক্লাসের `constructor`-এ `this` অ্যাক্সেস করার পূর্বে অবশ্যই প্যারেন্ট ক্লাসের `super()` কল করতে হবে।

ভুল পদ্ধতি: `SUPER()` না ডেকে `THIS` অ্যাক্সেস

✗ ERROR / INVALID

```
class Animal {
  constructor(name) {
    this.name = name;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    this.breed = breed;
  }
}
```

✗ REFERENCE ERROR

REFERENCE ERROR

ReferenceError: Must call super constructor in derived class before accessing 'this'

সঠিক সমাধান: প্যারেন্ট কনস্ট্রাক্টরে প্যারামিটার পাস করতে প্রথমে `super( ... )` কল করুন।

সঠিক পদ্ধতি: `SUPER(NAME)` কল

✓ CORRECT

```
class Animal {
  constructor(name) {
    this.name = name;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
}
```

24.46 Common Mistake: Private Field বাইরে Access

Private field (`#field`) ক্লাসের বাইরে থেকে সরাসরি ডট নোটেশনে অ্যাক্সেস করা যায় না। এটি করলে সিনট্যাক্স এরর দেয়।

ভুল পদ্ধতি: ক্লাসের বাইরে `PRIVATE FIELD` অ্যাক্সেস

✗ ERROR / INVALID

```
class User {
  #password = "12345";
}

const user = new User();
console.log(user.#password);
```

✗ SYNTAX ERROR

SYNTAX ERROR

SyntaxError: Private field '#password' must be declared in an enclosing class

সঠিক সমাধান: ক্লাসের নিজস্ব মেথড বা গিটারের মাধ্যমে প্রাইভেট ডেটা ড্যালিডেট বা এক্সেস করুন।

সঠিক পদ্ধতি: মেথডের মাধ্যমে এক্সেস

✓ CORRECT

```
class User {
  #password = "12345";

  verifyPassword(input) {
    return this.#password === input;
  }
}

const user = new User();
console.log(user.verifyPassword("12345")); // true
```

24.47 Common Mistake: Static Method Object দিয়ে Call

Static method সরাসরি ক্লাসের প্রপার্টি, এটি ইনস্ট্যান্স অবজেক্টে উত্তরাধিকারসূত্রে থাকে না। অবজেক্ট ইনস্ট্যান্স দিয়ে কল করলে TypeError দেয়।

ভুল পদ্ধতি: অবজেক্ট দিয়ে STATIC মেথড কল

✗ ERROR / INVALID

```
class Test {
  static hello() {
    console.log("Hello");
  }
}

const test = new Test();
test.hello();
```

✗ TYPE ERROR

TYPEERROR

TypeError: test.hello is not a function

সঠিক সমাধান: সরাসরি ক্লাসের নাম ধরে কল করুন: `Test.hello()`

সঠিক পদ্ধতি: ক্লাসের নাম ধরে কল

✓ CORRECT

```
class Test {
  static hello() {
    console.log("Hello");
  }
}

Test.hello(); // Hello
```

24.48 Common Mistake: Constructor-এ this

কনস্ট্রাক্টরের ভেতর `this.property = value` না লিখলে অবজেক্টের প্রপার্টি ইনিশিয়ালাইজ হয় না; শুধু লোকাল ভেরিয়েবল অ্যাসাইন হয়।

ভুল পদ্ধতি: THIS ছাড়া নাম অ্যাসাইন

✗ ERROR / INVALID

```
class Student {
  constructor(name) {
    name = name; // প্যারামিটার নিজের উপর অ্যাসাইন হচ্ছে, ইনস্ট্যান্সে নয়!
  }
}

const s = new Student("Shariar");
console.log(s.name); // undefined
```

► OUTPUT MISMATCH

UNDEFINED

undefined

সঠিক সমাধান: অবজেক্টের প্রপার্টি সেট করতে সর্বদা `this.name` ব্যবহার করুন।

সঠিক পদ্ধতি: THIS.NAME ব্যবহার

✓ CORRECT

```
class Student {
  constructor(name) {
    this.name = name; // ইনস্ট্যান্স অবজেক্টে প্রপার্টি সেট হলো
  }
}

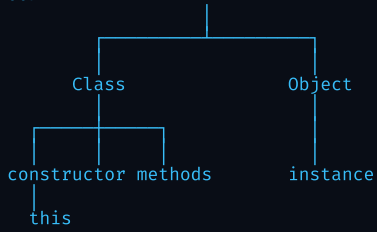
const s = new Student("Shariar");
console.log(s.name); // 'Shariar'
```

24.49 OOP Mental Model

এই পুরো chapter-টা এভাবে মনে রাখো:

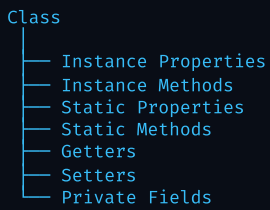
🕒 CONCEPT ARCHITECTURE

OOP



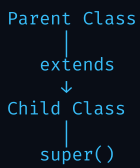
তারপর:

🕒 CONCEPT ARCHITECTURE



Inheritance:

🕒 CONCEPT ARCHITECTURE



OOP pillars:

► CONSOLE OUTPUT

OUTPUT

Encapsulation
Abstraction
Inheritance
Polymorphism

24.50 MUST KNOW

এই Chapter থেকে অবশ্যই জানতে হবে:

♦ 1. Class

JAVASCRIPT

JAVASCRIPT

```
class Student {}
```

♦ 2. Object

JAVASCRIPT

JAVASCRIPT

```
const student = new Student();
```

♦ 3. Constructor

JAVASCRIPT

JAVASCRIPT

```
constructor(name) {
  this.name = name;
}
```

♦ 4. this

Current instance/object reference।

♦ 5. Methods

JAVASCRIPT

JAVASCRIPT

```
study() {
  console.log("Studying");
}
```

♦ 6. Static

JAVASCRIPT

JAVASCRIPT

```
static hello() {}
```

Call:

JAVASCRIPT

JAVASCRIPT

```
ClassName.hello();
```

♦ 7. Getter

JAVASCRIPT

JAVASCRIPT

```
get name() {}
```

♦ 8. Setter

JAVASCRIPT

JAVASCRIPT

```
set name(value) {}
```

♦ 9. Private Field

JAVASCRIPT

JAVASCRIPT

```
#password
```

♦ 10. Inheritance

JAVASCRIPT

JAVASCRIPT

```
class Dog extends Animal {}
```

♦ 11. Parent constructor

JAVASCRIPT

JAVASCRIPT

```
super();
```

♦ 12. Method overriding

Child নিজের method দিয়ে parent method replace/override করতে পারে।

♦ 13. Polymorphism

একই method বিভিন্ন object-এ different behavior দিতে পারে।

♦ 14. Prototype connection

class syntax-এর নিচে JavaScript-এর prototype system কাজ করে।

24.Quick 🗨 Quick Cheat Sheet — Class & OOP Syntax Reference

JavaScript Class এবং Object-Oriented Programming এর দ্রুত রেফারেন্সের জন্য সম্পূর্ণ সিনটাক্স টেবিল:

Concept	Syntax
Class	<code>class User {}</code>
Constructor	<code>constructor() {}</code>
Object	<code>new User()</code>
Property	<code>this.name</code>
Method	<code>hello() {}</code>
Static	<code>static hello() {}</code>
Getter	<code>get name() {}</code>
Setter	<code>set name(value) {}</code>
Private	<code>#password</code>
Inheritance	<code>extends</code>
Parent constructor	<code>super()</code>
Parent method	<code>super.method()</code>
Instance check	<code>obj instanceof Class</code>

24.Lab 📁 Hands-On Practice Lab — 6 Real-World OOP Challenges

থিওরি এবং কনসেপ্ট আয়ত্ত করার পর বাস্তব প্রজেক্টে ক্লাস ব্যবহারের দক্ষতা যাচাই করার জন্য নিচে ৬টি প্র্যাকটিস প্রবলেমের বিস্তারিত সমাধান দেওয়া হলো:

Practice 1: Car Class — Brand, Model, Year & showInfo() Method Beginner

Problem Requirement: `Car` নামে একটি ক্লাস বানান যাতে `brand`, `model`, `year` প্রোপার্টি থাকবে এবং `showInfo()` মেথড গাড়ির সম্পূর্ণ বিবরণ প্রিন্ট করবে।

PRACTICE 1 MODEL SOLUTION

✓ CORRECT

```
class Car {
  constructor(brand, model, year) {
    this.brand = brand;
    this.model = model;
    this.year = year;
  }

  showInfo() {
    console.log(`Car Info: ${this.brand} ${this.model} (${this.year})`);
  }
}

// Verification
const myCar = new Car("Toyota", "Camry", 2024);
myCar.showInfo();
```

▶ CONSOLE OUTPUT

OUTPUT

Car Info: Toyota Camry (2024)

Practice 2: Student Class — Name, ID, Department & introduce() Method Beginner

Problem Requirement: `Student` নামে একটি ক্লাস বানান যাতে `name`, `id`, `department` প্রোপার্টি থাকবে এবং `introduce()` মেথড শিক্ষার্থীর পরিচয় প্রিন্ট করবে।

PRACTICE 2 MODEL SOLUTION

✓ CORRECT

```
class Student {
  constructor(name, id, department) {
    this.name = name;
    this.id = id;
    this.department = department;
  }

  introduce() {
    console.log(`Hello, I am ${this.name}, ID: ${this.id}, Department of ${this.department}.`);
  }
}

// Verification
const s1 = new Student("Rahim Ahmed", "CSE-1024", "Computer Science");
s1.introduce();
```

▶ CONSOLE OUTPUT

OUTPUT

Hello, I am Rahim Ahmed, ID: CSE-1024, Department of Computer Science.

Practice 3: Rectangle Class — Width, Height, area() & perimeter() Calculation Beginner

Problem Requirement: `Rectangle` নামে একটি ক্লাস বানান যার `width` ও `height` প্রোপার্টি থাকবে এবং আয়তক্ষেত্রের ক্ষেত্রফল ও পরিসীমা নির্ণয়ের জন্য `area()` ও `perimeter()` মেথড থাকবে।

PRACTICE 3 MODEL SOLUTION

✓ CORRECT

```
class Rectangle {
  constructor(width, height) {
    this.width = width;
    this.height = height;
  }

  area() {
    return this.width * this.height;
  }

  perimeter() {
    return 2 * (this.width + this.height);
  }
}

// Verification
const rect = new Rectangle(10, 5);
console.log("Area:", rect.area());
console.log("Perimeter:", rect.perimeter());
```

► CONSOLE OUTPUT

OUTPUT

```
Area: 50
Perimeter: 30
```

Practice 4: BankAccount with Private #balance, Deposit & Withdraw Validations Intermediate

Problem Requirement: `BankAccount` ক্লাস বানান যাতে ব্যালেন্স প্রাইভেট (`#balance`) থাকবে এবং `deposit(amount)`, `withdraw(amount)`, ও `getBalance()` মেথডের মাধ্যমে সুরক্ষিতভাবে লেনদেন করা যাবে।

PRACTICE 4 MODEL SOLUTION

✓ CORRECT

```
class BankAccount {
  #balance;

  constructor(initialBalance = 0) {
    this.#balance = initialBalance >= 0 ? initialBalance : 0;
  }

  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      console.log(`Deposited ${amount}. New balance: ${this.#balance}`);
    } else {
      console.log("Deposit amount must be positive.");
    }
  }

  withdraw(amount) {
    if (amount <= 0) {
      console.log("Invalid withdrawal amount.");
    } else if (amount > this.#balance) {
      console.log("Insufficient funds!");
    } else {
      this.#balance -= amount;
      console.log(`Withdrawn ${amount}. Remaining balance: ${this.#balance}`);
    }
  }

  getBalance() {
    return this.#balance;
  }
}

// Verification
const account = new BankAccount(500);
account.deposit(200);
account.withdraw(150);
console.log("Final Balance:", account.getBalance());
```

► CONSOLE OUTPUT

OUTPUT

```
Deposited $200. New balance: $700
Withdrawn $150. Remaining balance: $550
Final Balance: 550
```


Practice 5: Inheritance Architecture: Employee Base Class to Developer Derived Class Intermediate

Problem Requirement: `Employee` (name, salary) বেস ক্লাস থেকে `Developer` (language, code()) চাইল্ড ক্লাস তৈরি করুন এবং `super()` এর মাধ্যমে প্যারেন্ট প্রোপার্টি ইনিশিয়ালাইজ করুন।

PRACTICE 5 MODEL SOLUTION

✓ CORRECT

```
class Employee {
  constructor(name, salary) {
    this.name = name;
    this.salary = salary;
  }

  getDetails() {
    return `${this.name} earns $${this.salary}/year`;
  }
}

class Developer extends Employee {
  constructor(name, salary, language) {
    super(name, salary); // Call parent constructor
    this.language = language;
  }

  code() {
    console.log(`${this.name} is writing code in ${this.language}.`);
  }
}

// Verification
const dev = new Developer("Shariar", 95000, "JavaScript");
console.log(dev.getDetails());
dev.code();
```

▶ CONSOLE OUTPUT

OUTPUT

```
Shariar earns $95000/year
Shariar is writing code in JavaScript.
```

Practice 6: Mini E-commerce Architecture: Product, Cart, Order & User Interaction Advanced

Problem Requirement: একটি পূর্ণাঙ্গ অবজেক্ট-ওরিয়েন্টেড মিনি ই-কমার্স আর্কিটেকচার তৈরি করুন যাতে `Product`, `Cart`, `Order`, এবং `User` ক্লাসগুলো একে অপরের সাথে ইন্টারঅ্যাক্ট করে মোট মূল্য ও কার্ট প্রসেসিং সম্পন্ন করবে।

PRACTICE 6 MODEL SOLUTION

✓ CORRECT

```
class Product {
  constructor(id, name, price) {
    this.id = id;
    this.name = name;
    this.price = price;
  }
}

class Cart {
  constructor() {
    this.items = [];
  }

  addProduct(product, quantity = 1) {
    this.items.push({ product, quantity });
    console.log(`Added ${quantity}x ${product.name} to cart.`);
  }

  getTotal() {
    return this.items.reduce((sum, item) => sum + item.product.price * item.quantity, 0);
  }
}

class Order {
  constructor(user, cart) {
    this.user = user;
    this.items = [...cart.items];
    this.totalAmount = cart.getTotal();
    this.orderDate = new Date();
  }

  printReceipt() {
    console.log(`=== RECEIPT FOR ${this.user.name} ===`);
    this.items.forEach(i => console.log(`- ${i.product.name} x${i.quantity}: ${i.product.price * i.quantity}`));
    console.log(`Total: ${this.totalAmount}`);
  }
}

class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }
}

// Complete System Simulation
const user = new User("Tanjim", "tanjim@example.com");
const laptop = new Product(101, "MacBook Pro", 1999);
const mouse = new Product(102, "Magic Mouse", 79);

const cart = new Cart();
cart.addProduct(laptop, 1);
cart.addProduct(mouse, 2);

const order = new Order(user, cart);
order.printReceipt();
```

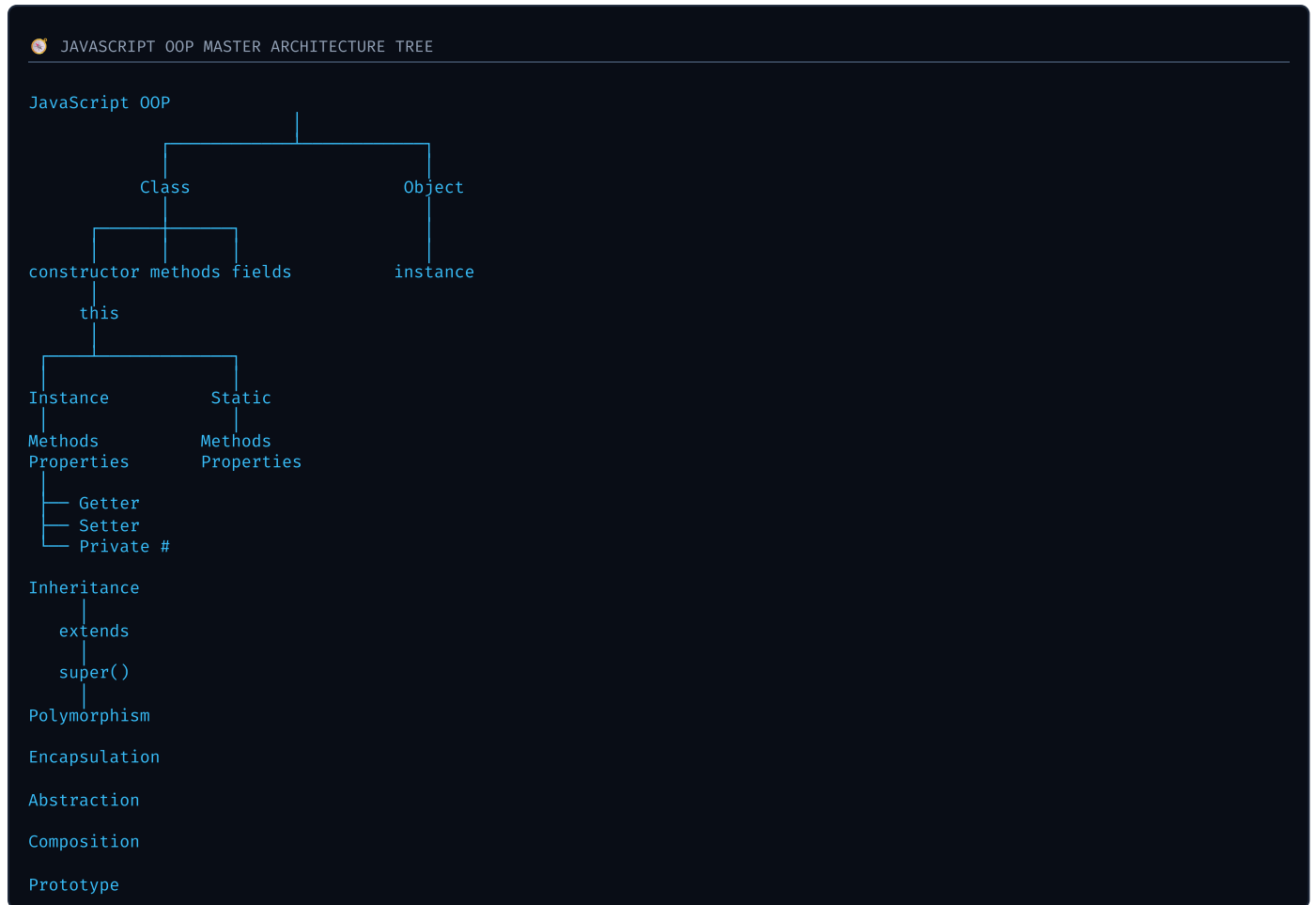
► CONSOLE OUTPUT

OUTPUT

```
Added 1x MacBook Pro to cart.
Added 2x Magic Mouse to cart.
=== RECEIPT FOR Tanjim ===
- MacBook Pro x1: $1999
- Magic Mouse x2: $158
Total: $2157
```

24.Map 🧠 Final Mental Map & Prototype Connection

JavaScript-এর সম্পূর্ণ Object-Oriented Programming ইকোসিস্টেম, ক্লাস ফিল্ডস, ইনহেরিট্যান্স এবং মেথড স্ট্রাকচারের সামগ্রিক ভিজুয়াল আর্কিটেকচার:



★ সবচেয়ে গুরুত্বপূর্ণ Connection

তুমি **Chapter 10-এ Objects**, **Chapter 11-এ Prototypes**, এবং **Chapter 24-এ Classes/OOP** পড়ার পর এই তিনটির পারস্পরিক যোগসূত্র এভাবে মনে রাখবে:

